Each ask is scoped to "trivial for DuckDB, removes a real hack on our side". Four of the five are pure additive public accessors — zero behavior change, can't affect existing DuckDB consumers. One (Ask #2) is a small new virtual method gated on read-only non-encrypted files, with a `return nullptr` default so callers that don't opt in see no difference.

Ask #1 — Five small public accessors

Problem

To walk the segment tree and read compression metadata from the extension side, we need five getters that today expose state already computed privately (or protected). Without them the extension can't walk storage internals without duplicating DuckDB's own logic.

Our current hack

We carry these as a local patch on our DuckDB submodule fork. That patch has been in place for several months. It works, but it means we ship a fork of DuckDB alongside the extension, which is fragile and unnecessary if the methods are upstream.

Why asking DuckDB helps

- Removes the need for Sirius to ship a DuckDB fork.
- No behavior change — all five are read-only accessors over state that's already computed.
- Future DuckDB changes that rename or reshape these private members are then visible to us as compile errors at upgrade time, rather than as silent patch-rebase conflicts.

Proposed API

Header	Accessor	What it e
`storage/data_table.hpp`	`GetRowGroupCollectionRef()`	`shared_ptr<RowGr` (the already-private `r`
`storage/table/row_group_collection.hpp`	`GetRowGroupsDirect()`	the segment tree (toda `GetRowGroups()` is
`storage/table/row_group.hpp`	`GetColumnDirect(StorageIndex)`	`ColumnData&` (today' protected)
`storage/table/column_segment.hpp`	`GetCompressionType()`	wraps `function.get`
`storage/table/standard_column_data.hpp`	`GetValidityData()`	the already-private val

Sketch:

```
// data_table.hpp
shared_ptr<RowGroupCollection>& GetRowGroupCollectionRef() { return row

// row_group_collection.hpp
shared_ptr<RowGroupSegmentTree> GetRowGroupsDirect() const { return Get
```

```
// row_group.hpp
ColumnData& GetColumnDirect(const StorageIndex &c) const { return GetCo

// column_segment.hpp
CompressionType GetCompressionType() const { return function.get().type

// standard_column_data.hpp
ValidityColumnData& GetValidityData() { return *validity; }
```

We're happy to open the PR and write tests.

Ask #2 — `BlockManager::GetDirectBlockPointer(block_id)` for read-only files

Problem

`BufferManager::Pin()` is the dominant CPU cost in our scan path. It takes a per-block mutex, copies the 256KB block payload from the underlying mmap into a `FileBuffer`, and bumps a refcount to guard against eviction. All three operations are necessary for writable DBs — but for a read-only `.duckdb` file, the block payload is already visible in the OS page cache, no eviction can happen, and the mutex just serializes parallel readers.

On a column-heavy scan the per-block `Pin()` cost dominates wall clock — the GPU is idle waiting for CPU to hand out pointers.

Our current hack

We mmap the `.duckdb` file ourselves inside the extension and compute block addresses arithmetically:

```
// Extension-side mmap bypass:
//   data_ptr = mmap_base
//            + BLOCK_START            // = 4096 * 3 (file-header const
//            + block_id * alloc_size  // from segment.block->GetBlockA
//            + hdr_size               // from segment.block->GetBlockH
//            + block_offset;
```

Validated once per process via `std::call_once` by memcmp'ing the first pinned block against the derived mmap pointer. Gated on read-only + non-encrypted. Performance-equivalent to what a patched DuckDB would give us.

Why asking DuckDB helps

1. **The `BLOCK_START = 4096 * 3` constant is a storage-format invariant** that only DuckDB can authoritatively publish. Today we're reverse-engineering it.
2. **Our memcmp validator is self-referential.** Both sides of the compare derive from the same pinned block, so the check only catches a hardcoded-constant-wrong-for-this-version failure, not a subtle layout shift to a different block. A silent format change (e.g. encryption header reshuffle) could slip through.
3. **Centralizes format knowledge inside DuckDB.** Today the arithmetic lives in both

places; keeping it one place is the right long-term shape.

Proposed API

```
// duckdb/storage/block_manager.hpp
class BlockManager {
  // ...
  virtual data_ptr_t GetDirectBlockPointer(block_id_t block_id) {
    return nullptr;  // default: not supported, caller falls back to Pi
  }
};

// SingleFileBlockManager overrides:
data_ptr_t SingleFileBlockManager::GetDirectBlockPointer(block_id_t blo
  if (!options.read_only || encryption_enabled) {
    return nullptr;
  }
  // lazy-mmap the file with atomic DCL init
  // return base + BLOCK_START + block_id * alloc_size + header_size
}
```

Gated to read-only, non-encrypted DBs. `return nullptr` default means no existing consumer sees any change. We're happy to open the PR; ~150 LOC across 5 files.

What we're NOT asking for

- Not asking to change `Pin()`. That's the hot write path; we don't want to touch it.
- Not asking for a buffer-pool bypass on writable DBs. The read-only gate is the whole safety story.
- Not asking for encryption support — `encryption_enabled` short-circuits to null and we fall back to `Pin()`.

Ask #3 — Expose the block-0 file-offset publicly

Problem

Even with Ask #2, our fallback path (for encrypted DBs, writable DBs, or older DuckDB versions) needs to know where block 0 begins in the file. That constant — the three headers DuckDB writes at the start of a `.duckdb` file, currently `4096 × 3 = 12288` bytes — is a storage-format invariant that today lives only in DuckDB's implementation.

Our current hack

Hardcoded `CANDIDATE_BLOCK_START = 4096 * 3` in extension source, memcmp-validated against a real pinned block.

As noted in Ask #2: the validator is self-referential. It catches a constant that's obviously wrong but not a subtle shift (e.g. the header count changing from 3 to 4, with block 0 happening to contain matching bytes at the wrong offset by coincidence).

Why asking DuckDB helps

- Single public const (or accessor) centralizes the layout knowledge where it lives: inside DuckDB.
- Removes a class of silent-breakage risk from the extension.
- Makes Ask #2's fallback path format-stable across DuckDB versions.

Proposed API

```
// duckdb/storage/block_manager.hpp — choose one:

// Cheapest: public const.
class BlockManager {
  static constexpr idx_t FILE_HEADER_SIZE = 4096 * 3;
};

// More future-proof: method that encapsulates any future layout shift.
class BlockManager {
  virtual idx_t GetFileOffsetForBlock(block_id_t block_id) const {
    return FILE_HEADER_SIZE + block_id * GetBlockAllocSize();
  }
};
```

Either works. The constant is simpler; the method is better if encryption or versioning ever shifts the offset.

Ask #4 — Per-column / per-row-group decompressed byte size

Problem

We need exact per-column decompressed size at **two pre-decode points**, before any GPU kernel has run:

1. **Query-prep time / batch sizing.** We walk every segment of every projected column of every row group to decide how many row groups fit in one scan task. For VARCHAR columns this uses `row_count × max_string_length` from `StringStats`, which is always an over-estimate.
2. **Upfront char-buffer allocation for string decode.** Our string decoder is a batched two-pass kernel: Pass 1 computes exact per-row lengths + CUB `ExclusiveSum` → exact offsets; Pass 2 gathers. To avoid an inter-pass sync, we allocate the output char buffer *before Pass 1 runs* using the same `max_string_length × row_count` upper bound. Pass 1 then writes into the pre-sized buffer.

In other words: yes, Pass 1 of our decoder does compute exact lengths — but that's too late to size the allocations that feed into Pass 1 itself, and it's way too late to decide batch layout at query prep.

Our current hack

- O(row_groups × columns × segments) walk in `check_viability()` on every query prep.
- VARCHAR over-allocation before Pass 1 (often 2–4× the real size, since

`max_string_length` ≫ `avg_length` on real workloads).

Why asking DuckDB helps

I found that DuckDB already computes this information internally — the CPU scan path sizes DataChunks from the same state. Exposing it would:

1. Replace the segment-tree walk with a single accessor per column per row group.
2. Let us allocate VARCHAR output buffers closer to their true size, freeing GPU memory for more concurrent scan tasks and reducing OOM risk on wide-table workloads.

The specific number we need for VARCHAR is the **total decompressed-char-bytes** — sum of string lengths — not max × count. If `StringStats` already tracks something closer to the sum than the max, exposing that would cover this ask.

Proposed API

```
// duckdb/storage/table/column_data.hpp
idx_t ColumnData::GetDecompressedSize(idx_t row_start, idx_t row_end) c

// OR at row-group granularity:
// duckdb/storage/table/row_group.hpp
idx_t RowGroup::GetColumnDecompressedSize(const StorageIndex &col_idx)
```

Either is fine. We can consume whichever granularity matches DuckDB's existing internal bookkeeping.

Ask #5 — Publish compression-format header structs as public POD types

Problem

We decode DuckDB's on-disk compressed blocks directly on the GPU. To do that, we need to know the layout of the per-compression-type headers embedded at the start of each segment's block data. Today those layouts are *internal* to DuckDB, so we carry duplicated copies on our side:

```
// Duplicated in our CUDA source from DuckDB internals:
struct dict_header_t {
  uint32_t dict_size;
  uint32_t dict_end;
  uint32_t index_buffer_offset;
  uint32_t index_buffer_count;
  uint32_t bitpacking_width;
};

struct fsst_header_t {
  uint32_t dict_size;
  uint32_t dict_end;
  uint32_t bitpacking_width;
  uint32_t fsst_symbol_table_offset;
```

```
};
```

We also carry format knowledge for the bitpacking per-2048-row metadata group encoding (24-bit mode + 8-bit data_offset, entries stored backwards from `metadata_end` at segment byte 0), the RLE layout (8-byte count_offset, values, then uint16 counts), and so on. None of this is documented in a public header — we reverse-engineered the layouts from the DuckDB source and hold them as a contract.

Our current hack

Static struct declarations in our CUDA kernels, parsed via `memcpy` from the block byte stream. Field order and types match DuckDB's current implementation. No version check — we trust that the layout hasn't changed.

Why asking DuckDB helps

This is the ask most likely to prevent silent correctness bugs.

We've already hit one: our int64 bitpacking unpack for the edge case where a single value spans 3 consecutive 32-bit words was wrong for certain width/offset combinations, causing silent bad answers on timestamp columns. The bug took a day to isolate because the data looked right almost all the time.

That class of bug is exactly what a public format header would prevent — not because we'd catch the edge case automatically, but because any DuckDB engineer reviewing their own public struct knows it has downstream consumers and thinks twice before changing it.

Proposed API

```
// duckdb/storage/compression/storage_format.hpp — new public header
namespace duckdb::storage_format {

constexpr uint32_t STORAGE_FORMAT_VERSION = 1;  // bump on on-disk layo

struct dictionary_compression_header_t {
  uint32_t dict_size;
  uint32_t dict_end;
  uint32_t index_buffer_offset;
  uint32_t index_buffer_count;
  uint32_t bitpacking_width;
};

struct fsst_compression_header_t {
  uint32_t dict_size;
  uint32_t dict_end;
  uint32_t bitpacking_width;
  uint32_t fsst_symbol_table_offset;
};

// Bitpacking per-2048-row metadata group encoding:
//   - first 8 bytes of segment = end-of-metadata offset (uint64)
//   - metadata entries are consecutive uint32s stored backwards from t
//   - each entry: (mode << 24) | (data_offset & 0x00FFFFFF)
```

```
//
// RLE segment layout:
//   - first 8 bytes of segment = rle_count_offset (uint64)
//   - values: T[entry_count] starting at byte 8
//   - counts: uint16_t[entry_count] starting at rle_count_offset

} // namespace duckdb::storage_format
```

Consumers can `static_assert(storage_format::STORAGE_FORMAT_VERSION == 1)` to fail loudly at compile time on upgrade. Piecemeal is also fine — just the dict and FSST headers as public POD types would cover the most common codecs.

Summary

Ask	DuckDB-side cost	What it removes from the extension
1. Five public getters	5 one-liners across 5 headers	Need to ship a DuckDB fork
2. `GetDirectBlockPointer`	~150 LOC, 5 files, read-only gate	The dominant CPU cost of our scan; reverse-engineered mmap math
3. Block-0 file offset	1–3 lines	Hardcoded `4096*3` constant + self-referential validator
4. Decompressed byte size	1 accessor over existing internal state	Per-query segment-tree walk + VARCHAR over-allocation
5. Public format headers	New header, POD structs	Silent-correctness-bug risk from format drift (we've hit one)

Asks 1, 3, 4, 5 are pure additive public accessors — zero behavior change, no impact on any existing DuckDB user. Ask 2 is the only one with real code, and the entire thing is gated on `read_only && !encrypted` with a `return nullptr` default — existing callers are unaffected.

We're happy to open each PR ourselves and maintain them. We'd just like a nod on the design (particularly #2 and the shape of #4 and #5) before spending the time.